

Merge Sort



Time Complexity of Bubble Sort

$O(n^2)$

$O(n \log n)$

$O(n)$

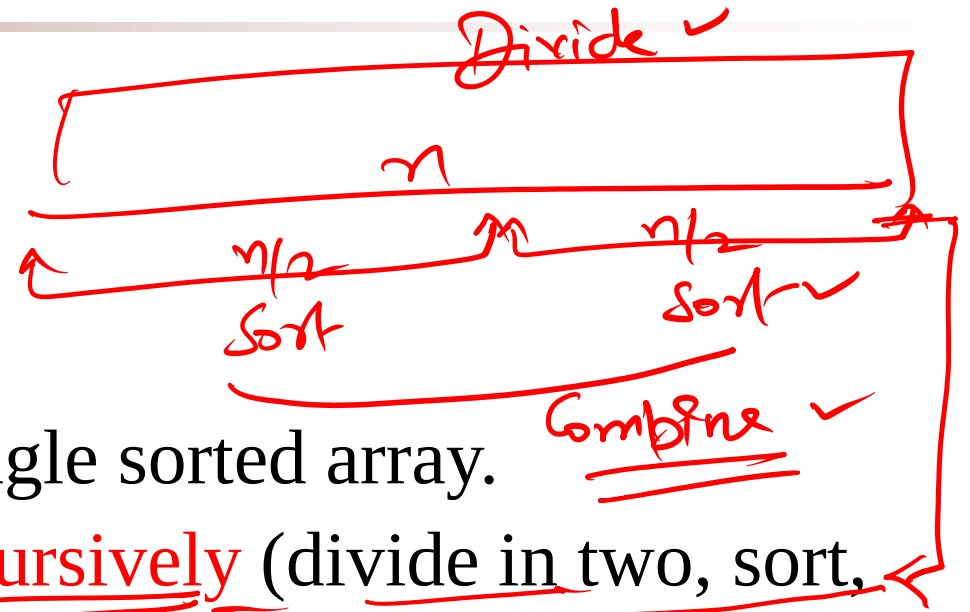
$O(n^2 \log n)$

Start the presentation to activate live content

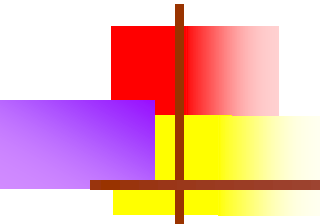
If you see this message in presentation mode, install the add-in or get help at Pollev.com/app

Basic Idea of Merge sort

- Divide the array into two halves.
- Sort the two sub-arrays (How?)
- Merge the two sorted sub-arrays into a single sorted array.
- Step 2 (sorting the sub-arrays) is done recursively (divide in two, sort, merge) until the array has a single element (base condition of recursion).



until ①
divide - sort → merge



Mergesort Example

- Let us take an example of an array with 8 elements

[16 9 12 48 30 11 65 22]

- We shall split it recursively in two parts till the array contains only a single element.
- Then we shall merge those two subarrays and move to higher sub-arrays
- In this case, we know that after 3 splits , the sub array will contain single elements.

$$2^3 = 8$$

[16 9 12 48 30 11 65 22] ← Input array

Left [16 9 12 48 | 30 11 65 22] Right

Split 1:

left: [16 9 | 12 48]

right: [30 11 | 65 22]

Split 2:

[16 | 9] [12 | 48]

[30 | 11] [65 | 22]

Split 3:

[16] [9] [12] [48]

[30] [11] [65] [22]

no more splits possible

Merge1: ✓

Merge2:

[9 16]

[12 48]

Now both subarrays at split 2 have been sorted so we can merge these

[9 12 16 48] ✓

Merge3:

[9 16]

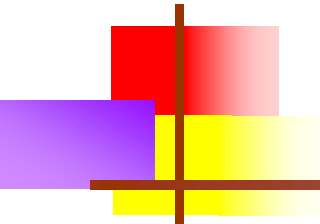
[12 48]

[9 12 16 48] ✓

④
subarrays
of 1 element

division

merging



Now take up the right half of Split 1

Take right half [30 11 | 65 22]
 [30 | 11] [65 | 22]
 [30] [11] [65] [22]

Merge4: [11 30]

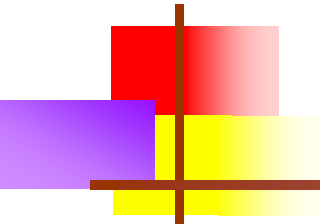
Merge5: [22 65]

Merge6: [11 22] [30 65]

⇒ two sorted sub array
of four elements
each

Merge7: ^{left} [9 12 16 48] and ^{right} [11 22 30 65]
 [9 11 12 16 22 30 48 65]

Sorting 8 elements required 7 merge steps
Sorting n elements requires n - 1 merge steps



Use mergesort on

[23 4 89 12 | 45 9 78 10]

[23 4 | 89 12] [45 9 | 78 10]

[23 | 4] [89 | 12] [45 9] [78 10]

[23] [4] [89] [12] [45] [9] [78] [10]

[4 23] Proceed in same manner as before

[4 23] [12 89] [9 45] [10 78]

[4 12 23 89] [9 10 45 78]

[4 9 10 12 23 45 78 89]

Merge sort - 1

- Mergesort needs a helper array to merge two sorted arrays.

```
public class Mergesort {  
    private int []A;  
    private int []aux;  
    private int n;  
    public void sort(int []A, n)  
    {
```

```
        this.A = A;  
        this.aux = new int[n];  
        mergesort(0, n - 1);  
    }
```

Initialization

first last element

Merge sort - 2

division

- Recursively keeps on dividing the array in two halves

```
private void mergesort(int low, int high) {  
    if (low < high) {
```

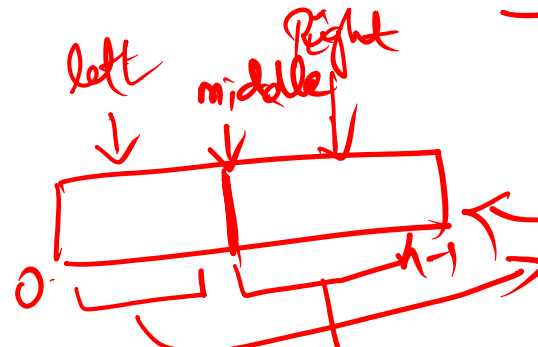
```
        int middle = (low + high) / 2;
```

```
        mergesort(low, middle);
```

```
        mergesort(middle + 1, high);
```

```
        // Combine both the sorted subarrays  
        merge(low, middle, high);
```

Combine

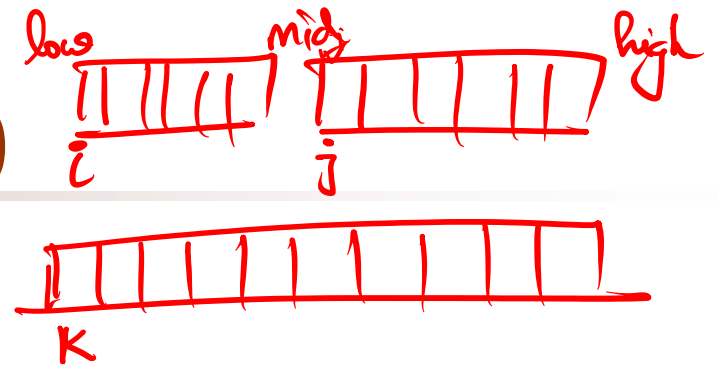


until

*low = high
only one element left*

division

Merge sort – 3 (the merge part)



- A is copied into an auxiliary array.
- Pointers i and j are set for two sub aux arrays and pointer k is set for the array which collects elements after merging.

```
private void merge(int low, int middle, int high) {
```

```
    // Copy contents of A into aux array
```

```
    for (int i = low; i <= high; i++) {
```

```
        aux[i] = A[i];
```

```
    }
```

```
    int i = low;
```

```
    int j = middle + 1;
```

```
    int k = low;
```

```
    // Copy the smallest element from either sub arrays into A
```

— first sorted array

second sorted array

⇒ Third Array / Output

Merge sort - 4

- Smaller of the two elements from the sub arrays are transferred to the merged array

```
while (i <= middle && j <= high) {  
    if (aux[i] <= aux[j]) {  
        A[k] = aux[i];  
        i++;  
    } else {  
        A[k] = aux[j];  
        j++;  
    }  
    k++; — output array pointer  
}
```

first array last element (pointing to middle)
second array last element (pointing to high)

Merge sort - 5

- If all the elements of one subarray have been put in array A, remaining elements of other array are simply copied into A.

// Copy the rest of the elements

```
while (i <= middle) {  
    A[k] = aux[i];  
    k++;  
    i++;  
}
```

```
while (J <= high) {  
    A[k] = aux[i];  
    J++;  
    k++;  
}
```

Time Complexity of insertion sort at Average case

$O(n^2)$

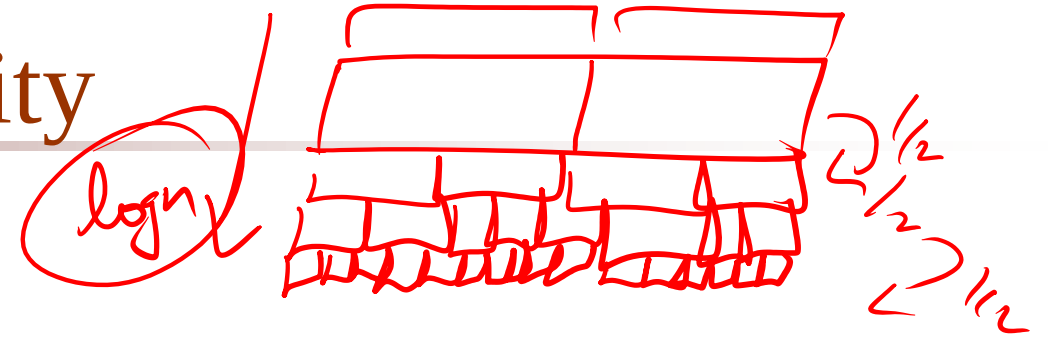
$O(n)$

$O(n \log n)$

Start the presentation to activate live content

If you see this message in presentation mode, install the add-in or get help at Pollev.com/app

Mergesort Time complexity



- Like binary search array is divided in two parts at every stage, so work done would be of order $\log n$.
- At every stage each element of the array is copied and may be processed. This is bound to be of order n .
- Thus intuitively mergesort needs $O(n \log n)$ operations.

Best, worst & average

Mergesort Time Complexity

two problem $n/2$ size

each time require n

$$T(n) = 2T(n/2) + n, \quad T(1) = 1$$

$$T(n) = 2^1 T(n/2^1) + 1n$$

$$T(n) = 2T(n/2) + n$$

$$T(n) = 2^2 T(n/2^2) + 2n$$

$$= 4 T(n/4) + 2n$$

$$T(n) = 2^3 T(n/2^3) + 3n$$

$$= 8 T(n/8) + 3n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$T(n) = 2^k T(n/2^k) + nk$$

$(n \log n)$

$$\frac{n}{2^k} = 1$$
$$k = \log n$$

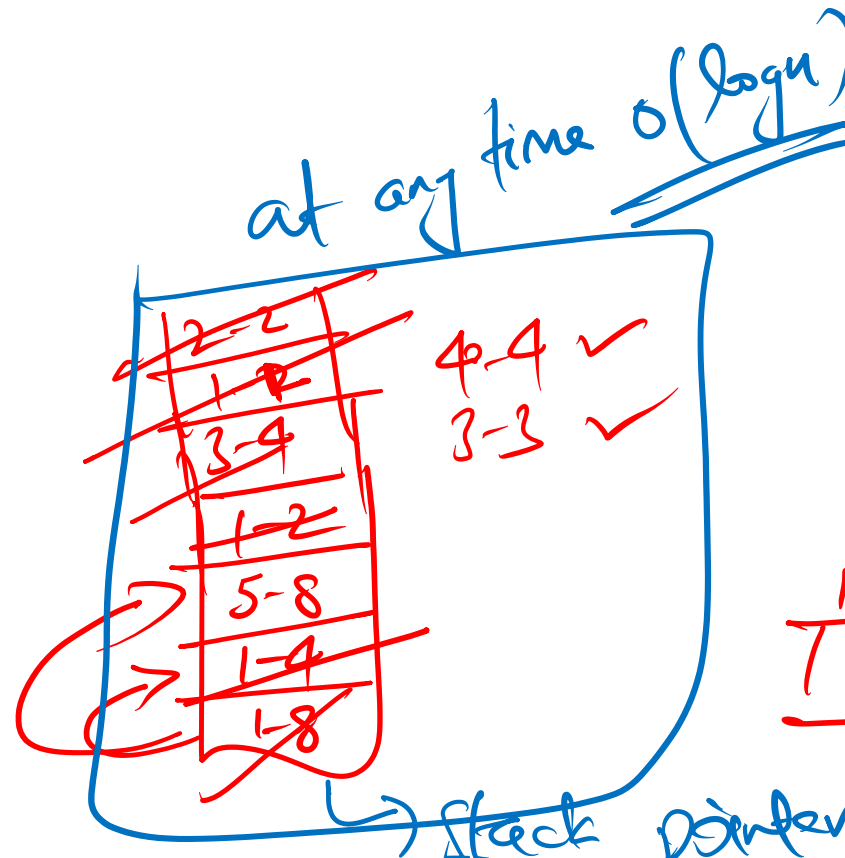
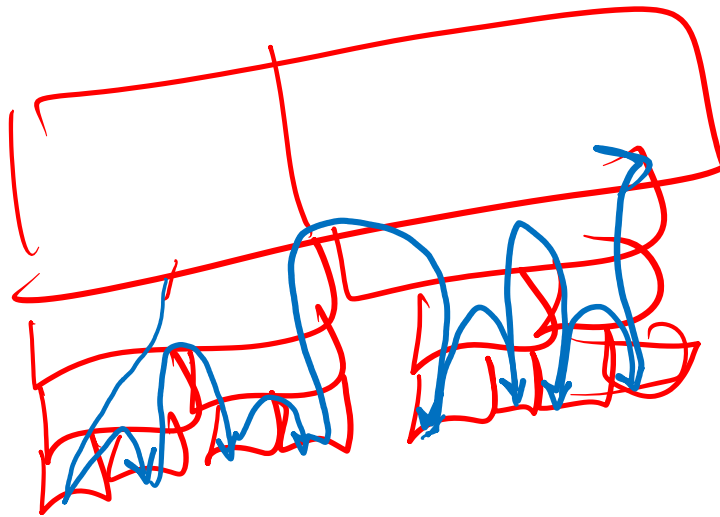
To use Boundary condition let us try to bring it to form $T(1)$

$$T(n) = n T(1) + n \log n$$

$$= O(n \log n)$$

MergeSort Space Complexity

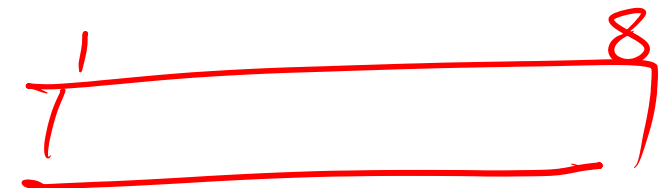
- N space for auxiliary array for merging the array $O(n)$.
- $O(\log n)$ for recursion part. ?
- Total = $O(n + \log n)$



recursion
require
space

```

mergeSort() {
  mergeSort()
  mergeSort()
  merge()
}
    
```



Space complexity of Selection Sort

max/min $\Rightarrow$ dest-insert

$O(n)$

$O(n \log n)$

$O(n^2)$

$O(1)$

Start the presentation to activate live content

If you see this message in presentation mode, install the add-in or get help at Pollev.com/app